# Programação distribuída

## 1º semestre de 2026

## Processos

**Tema #01**

Professor Marcelo Eustáquio

**Processos** são a engrenagem invisível que dá vida ao código, transformando instruções estáticas em execução dinâmica, coordenando recursos do sistema e tornando possível a concorrência que sustenta a computação moderna.

***Marcelo Eustáquio***

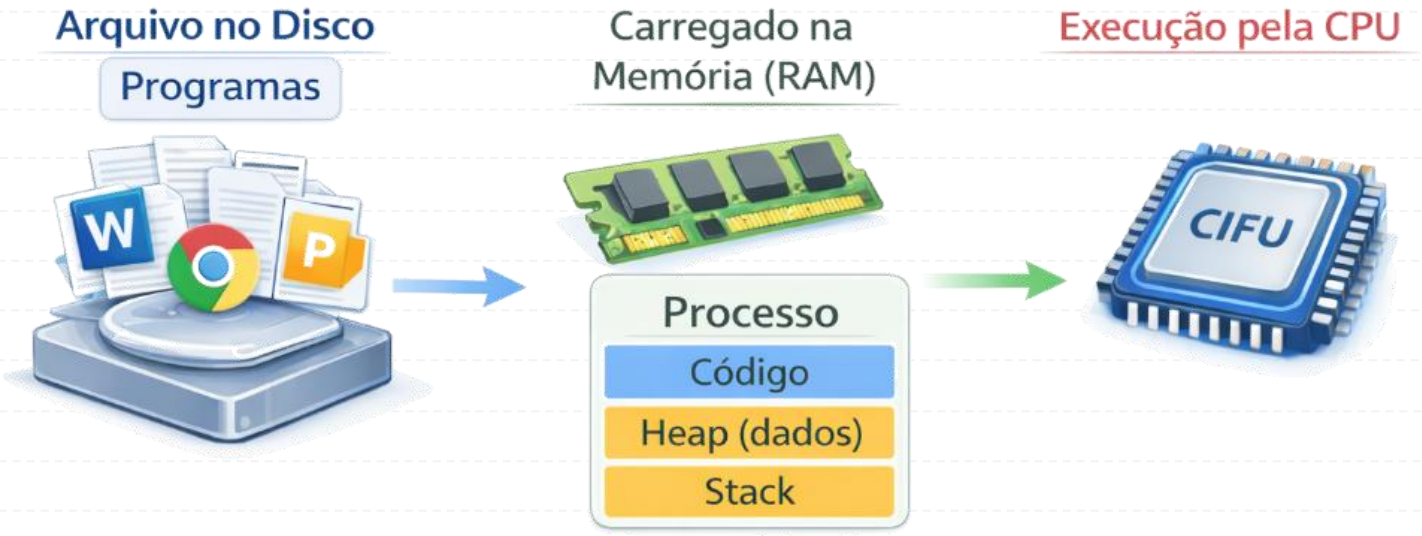
# Processos

**Programa** é é um conjunto de instruções escritas em uma linguagem de programação, armazenados em um arquivo e destinado a realizar uma tarefa quando executado pelo sistema operacional.

Programas são caracterizados pelo código-fonte ou compilado, linguagem formal (C, Java, Python, Assembler e outras), independe de execução, não possui estado ativo. Ou seja, programa é **estático**.

# Processos

**Processo** é definido como *programa em execução*, carregado na memória principal ou primária e gerenciado pelo sistema operacional. Assim, é um conceito complementar ao conceito de **programa**.




Processos estão ativos e sendo executados pela CPU, possuem PID (Process ID), possuem espaço de memória próprio, possui estado de execução e utiliza recursos do sistema (CPU, memória, arquivos).

# Processos

Na ciência da computação, **processo** é uma instância de um programa em execução em um sistema operacional, podendo ser considerada uma entidade ativa que inclui o código do programa, seus dados, recursos alocados e informações de execução, como registradores e contador de programa.

| | | | | | |
|---|---|---|---|---|---|
| Google Chrome (9) | | 0,3% | 0,1 MB/s | 478,2 MB | 0,1 Mbps |
| Google Chrome | | 0,1% | 0,1 MB/s | 285,8 MB | 0 Mbps |
| Google Chrome | | 0,1% | 0,1 MB/s | 13,0 MB | 0,1 Mbps |
| Google Chrome | | 0,1% | 0 MB/s | 6,3 MB | 0 Mbps |
| Google Chrome | Modo de ... | 0% | 0 MB/s | 21,8 MB | 0 Mbps |
| Google Chrome | | 0% | 0 MB/s | 6,7 MB | 0 Mbps |
| Google Chrome | | 0% | 0 MB/s | 134,5 MB | 0 Mbps |
| Google Chrome | | 0% | 0 MB/s | 1,0 MB | 0 Mbps |
| Google Chrome | | 0% | 0 MB/s | 2,8 MB | 0 Mbps |
| Google Chrome | Modo de ... | 0% | 0 MB/s | 6,3 MB | 0 Mbps |

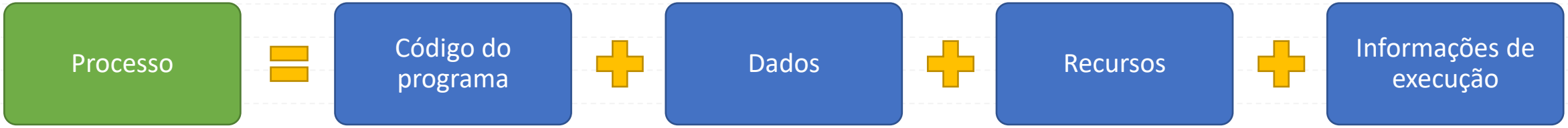
**Processo** é uma instância de um programa em execução em um sistema operacional, ou seja, uma entidade ativa que inclui o **código** do programa, seus **dados**, **recursos** alocados e **informações** de execução, como registradores e contador de programa.

***Marcelo Eustáquio***

# Processos

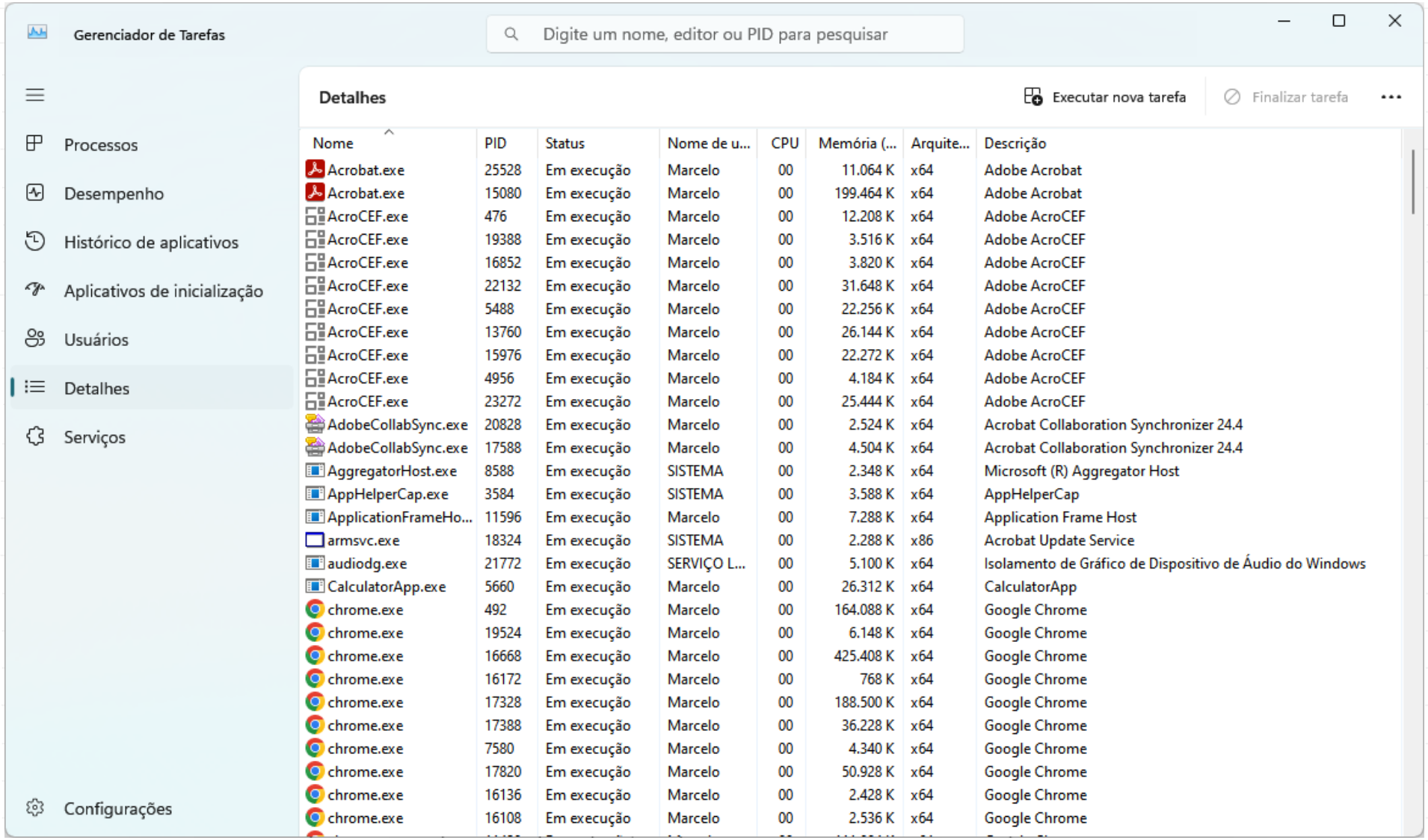
Na ciência da computação, **processo** é uma instância de um programa em execução em um sistema operacional, podendo ser considerada uma entidade ativa que inclui o código do programa, seus dados, recursos alocados e informações de execução, como registradores e contador de programa.

Ou seja:




**Importante**

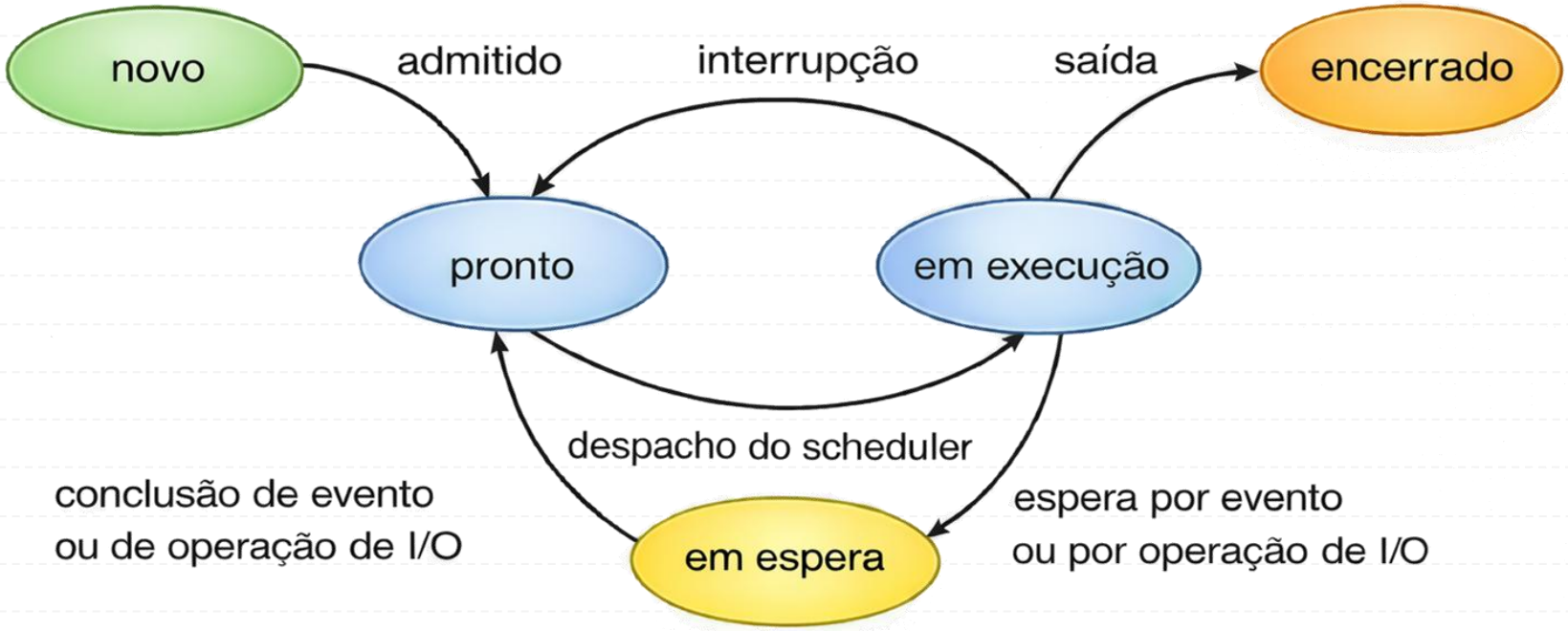
Processo é um programa em execução em uma máquina qualquer e é identificado pelo seu **PID** *(Process Identifier).*




| Nome | PID | Status | Nome de u... | CPU | Memória (... | Arquite... | Descrição |
|---|---|---|---|---|---|---|---|
| Acrobat.exe | 25528 | Em execução | Marcelo | 00 | 11.064 K | x64 | Adobe Acrobat |
| Acrobat.exe | 15080 | Em execução | Marcelo | 00 | 199.464 K | x64 | Adobe Acrobat |
| AcroCEF.exe | 476 | Em execução | Marcelo | 00 | 12.208 K | x64 | Adobe AcroCEF |
| AcroCEF.exe | 19388 | Em execução | Marcelo | 00 | 3.516 K | x64 | Adobe AcroCEF |
| AcroCEF.exe | 16852 | Em execução | Marcelo | 00 | 3.820 K | x64 | Adobe AcroCEF |
| AcroCEF.exe | 22132 | Em execução | Marcelo | 00 | 31.648 K | x64 | Adobe AcroCEF |
| AcroCEF.exe | 5488 | Em execução | Marcelo | 00 | 22.256 K | x64 | Adobe AcroCEF |
| AcroCEF.exe | 13760 | Em execução | Marcelo | 00 | 26.144 K | x64 | Adobe AcroCEF |
| AcroCEF.exe | 15976 | Em execução | Marcelo | 00 | 22.272 K | x64 | Adobe AcroCEF |
| AcroCEF.exe | 4956 | Em execução | Marcelo | 00 | 4.184 K | x64 | Adobe AcroCEF |
| AcroCEF.exe | 23272 | Em execução | Marcelo | 00 | 25.444 K | x64 | Adobe AcroCEF |
| AdobeCollabSync.exe | 20828 | Em execução | Marcelo | 00 | 2.524 K | x64 | Acrobat Collaboration Synchronizer 24.4 |
| AdobeCollabSync.exe | 17588 | Em execução | Marcelo | 00 | 4.504 K | x64 | Acrobat Collaboration Synchronizer 24.4 |
| AggregatorHost.exe | 8588 | Em execução | SISTEMA | 00 | 2.348 K | x64 | Microsoft (R) Aggregator Host |
| AppHelperCap.exe | 3584 | Em execução | SISTEMA | 00 | 3.588 K | x64 | AppHelperCap |
| ApplicationFrameHo... | 11596 | Em execução | Marcelo | 00 | 7.288 K | x64 | Application Frame Host |
| armsvc.exe | 18324 | Em execução | SISTEMA | 00 | 2.288 K | x86 | Acrobat Update Service |
| audiodg.exe | 21772 | Em execução | SERVIÇO L... | 00 | 5.100 K | x64 | Isolamento de Gráfico de Dispositivo de Áudio do Windows |
| CalculatorApp.exe | 5660 | Em execução | Marcelo | 00 | 26.312 K | x64 | CalculatorApp |
| chrome.exe | 492 | Em execução | Marcelo | 00 | 164.088 K | x64 | Google Chrome |
| chrome.exe | 19524 | Em execução | Marcelo | 00 | 6.148 K | x64 | Google Chrome |
| chrome.exe | 16668 | Em execução | Marcelo | 00 | 425.408 K | x64 | Google Chrome |
| chrome.exe | 16172 | Em execução | Marcelo | 00 | 768 K | x64 | Google Chrome |
| chrome.exe | 17328 | Em execução | Marcelo | 00 | 188.500 K | x64 | Google Chrome |
| chrome.exe | 17388 | Em execução | Marcelo | 00 | 36.228 K | x64 | Google Chrome |
| chrome.exe | 7580 | Em execução | Marcelo | 00 | 4.340 K | x64 | Google Chrome |
| chrome.exe | 17820 | Em execução | Marcelo | 00 | 50.928 K | x64 | Google Chrome |
| chrome.exe | 16136 | Em execução | Marcelo | 00 | 2.428 K | x64 | Google Chrome |
| chrome.exe | 16108 | Em execução | Marcelo | 00 | 2.536 K | x64 | Google Chrome |

# Estados de um processo

O estado de um processo descreve sua *situação atual dentro do ciclo de vida* em um sistema operacional e indica o que o processo está fazendo ou aguardando, e é uma das informações fundamentais que o sistema operacional usa para gerenciar a execução de processos.



# Estados de um processo

Os sistemas operacionais organizam a execução dos programas por meio de estados bem definidos, que representam as diferentes fases do ciclo de vida de um processo. Cada estado indica a situação atual do programa em relação ao uso da CPU e aos recursos do sistema, permitindo ao escalonador gerenciar de forma eficiente a execução das tarefas.

**1)** No **estado novo** (**new**): Processo recém-criado, ainda não admitido para execução.

**2)** No **estado pronto** (**ready**): Processo apto a executar, aguardando a liberação da CPU.

**3)** No **estado executando** (**running**): Processo que está utilizando a CPU no momento.

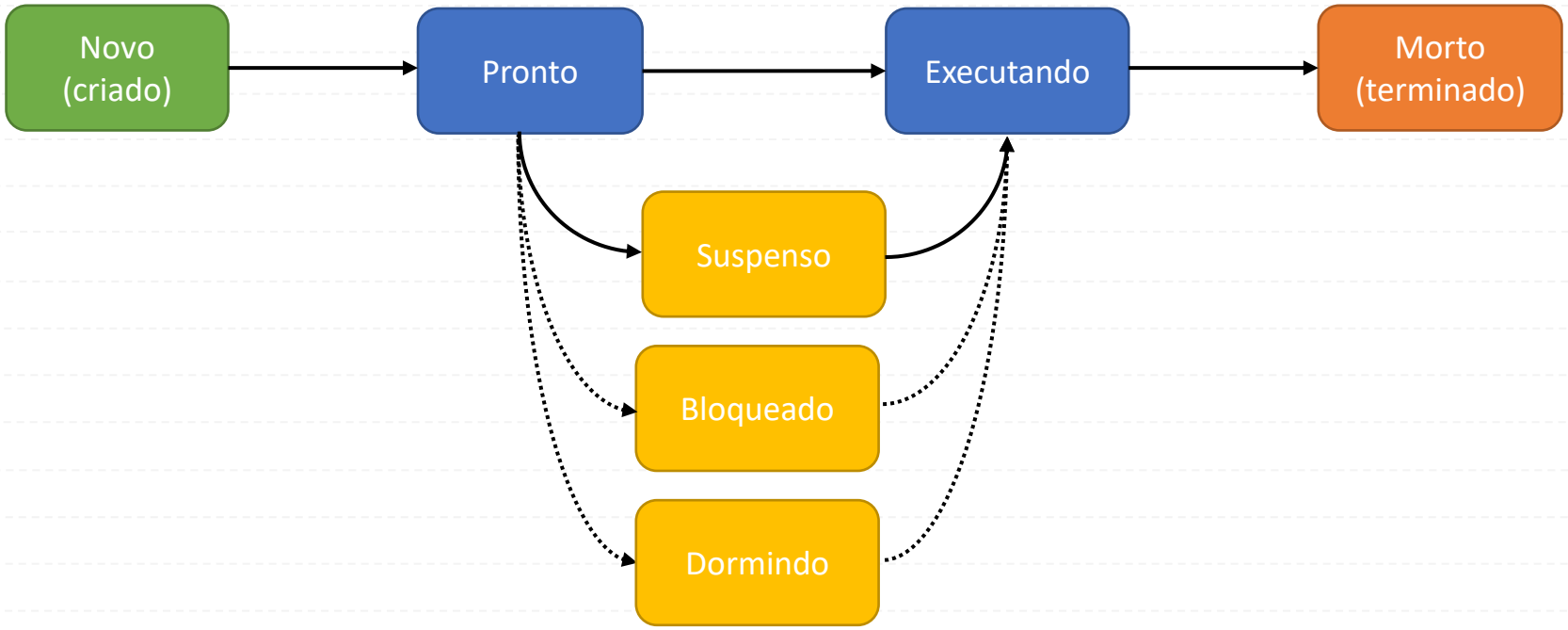
**4)** No **estado esperando** (**waiting**): Processo aguardando algo.

**5)** No **estado encerrado** (**terminated**): Processo que finalizou sua execução e foi concluído pelo sistema.

# Estados de um processo

Um exemplo simples está no processo de abertura e uso de um editou de textos para escrever e salvar um trabalho (Esse ciclo pode ocorrer várias vezes durante a execução do programa, alternando principalmente entre em execução, em espera e pronto, até que o usuário o encerre):

1) Novo: Você clica duas vezes no ícone do editor de texto. O sistema operacional *cria o processo* do programa e aloca os recursos iniciais (memória, estruturas de controle).
2) Pronto: O programa já foi carregado na memória e está apto a executar, mas ainda *está aguardando sua vez* na CPU, junto com outros processos do sistema.
3) Em execução: O *escalonador escolhe o editor de texto*, e ele passa a usar a CPU. A interface aparece na tela e você começa a digitar.
4) Em espera: Ao clicar em "Salvar", o programa precisa gravar o arquivo no disco. Durante essa operação de E/S (entrada/saída), ele *fica aguardando a conclusão* da escrita no armazenamento.
5) Pronto (novamente): Quando o salvamento termina, o *processo volta para o estado de pronto*, aguardando nova liberação da CPU.
6) Encerrado: Você fecha o editor. O sistema *finaliza o processo*, libera a memória e demais recursos utilizados.


Novo
(criado)
Pronto
Executando
Morto
(terminado)
Suspenso
Bloqueado
Dormindo

Em Python, é possível listar os processos ativos no sistema operacional utilizando o módulo **psutil**, que fornece uma interface para monitorar e gerenciar processos e outros recursos do sistema.

Gerenciamento de **Processos** (lista todos os PIDs ativos, itera sobre processos ativos, e cria objeto de processo); **CPU** (dispões de funções para monitoramento do processador); **Memória** que retornam informações como total, disponível, usada e virtual); **Disco** (partições, uso de disco, I/O de disco); **Rede** (interfaces de rede, status das interfaces; conexões ativas e estatísticas de tráfego); e **Sistema** (tempo de inicialização, usuários logados, temperatura (quando suportado) e bateria (em notebooks)).

# Processos

Escreva um programa que recebe o PID de um processo e exibe seu estado atual (ex.: em execução, parado, suspenso).

```python
import psutil

def estado_processo(pid):
    try:
        processo = psutil.Process(pid)
        estado = processo.status()
        return estado
    except psutil.NoSuchProcess:
        return "Processo não encontrado"

# Exemplo de uso

pid = int(input("Digite o PID do processo: "))
estado = estado_processo(pid)
print(f"O estado do processo com PID {pid} é: {estado}")
```

# Processos

Na ciência da computação, **processo** é uma instância de um programa em execução em um sistema operacional. Ele é uma entidade ativa que inclui o código do programa, seus dados, recursos alocados e informações de execução, como registradores e contador de programa.

```
import psutil

# Lista todos os processos ativos

for proc in psutil.process_iter(['pid', 'name']):
    try:
        print(f"PID: {proc.info['pid']}, Nome: {proc.info['name']} ")
    except (psutil.NoSuchProcess, psutil.AccessDenied, psutil.ZombieProcess):
        pass
```

**psutil.process_iter()** retorna um iterador para todos os processos ativos no sistema e **['pid', 'name']** especifica quais atributos do processo devem ser obtidos, evitando sobrecarga desnecessária.

# Processos

```
try:
    print(f"PID: {proc.info['pid']}, Nome: {proc.info['name']} ")
except (psutil.NoSuchProcess, psutil.AccessDenied, psutil.ZombieProcess):
    pass
```

No bloco **try**, o código tenta acessar as informações do processo usando o dicionário **proc.info**, quais sejam: 1) proc.info['pid'], que recupera o PID do processo; e 2) proc.info['name'] , que recupera o nome do processo. Essas informações são formatadas e impressas na tela usando uma f-string.

Durante a iteração sobre os processos, foram consideradas as exceções **psutil.NoSuchProcess** (em que o processo pode ter terminado entre o momento em que foi detectado e o momento em que suas informações foram acessadas), **psutil.AccessDenied** (em que o usuário que está executando o script pode não ter permissão para acessar informações de determinados processos) e **psutil.ZombieProcess** (o processo pode estar em estado "zumbi“, um estado especial em sistemas Unix-like onde o processo terminou, mas ainda está presente na tabela de processos até que seu status seja lido pelo processo pai).

# Processos

Outros parâmetros possíveis são:

1. pid: ID do processo.
2. name: Nome do executável do processo.
3. exe: Caminho completo do executável do processo.
4. cmdline: Linha de comando usada para iniciar o processo.
5. status: Estado atual do processo (e.g., running, sleeping, stopped, zombie).

# Processos

Modifique o código-fonte a seguir para mostrar apenas os processos que estão em execução (running).

```
import psutil

# Lista todos os processos ativos

for proc in psutil.process_iter(['pid', 'name']):
    try:
        print(f"PID: {proc.info['pid']}, Nome: {proc.info['name']} ")
    except (psutil.NoSuchProcess, psutil.AccessDenied, psutil.ZombieProcess):
        pass
```

**Importante**

O estado atual de um processo é acessado a partir do atributo status, que pode ser "running", "sleeping", "stopped" ou "zombie".

# Processos

Modificando o código-fonte a seguir para mostrar apenas os processos que estão em execução (running). Para tal, no interior do loop, será acrescentado um “if” para verificar se o status do processo considerado é igual a STATUS_RUNNING.

```
import psutil

# Lista todos os processos ativos

for proc in psutil.process_iter(['pid', 'name', 'status']):
    try:
        # Verifica se o processo está em execução (running)
        if proc.info['status'] == psutil.STATUS_RUNNING:
            print(f"PID: {proc.info['pid']}, Nome: {proc.info['name']}, Estado:
{proc.info['status']}")
    except (psutil.NoSuchProcess, psutil.AccessDenied, psutil.ZombieProcess):
        # Ignora exceções para processos problemáticos
        pass
```

# Processos

Outros parâmetros possíveis são:

| | |
|---|---|
| 6 | username: Nome do usuário que iniciou o processo. |
| 7 | create_time: Horário em que o processo foi iniciado (em timestamp Unix). |
| 8 | cpu_percent: Percentual de uso da CPU pelo processo. |
| 9 | memory_percent: Percentual de uso da memória pelo processo. |
| 10 | memory_info: Informações detalhadas sobre uso de memória (e.g., RSS, VMS). |

**RSS** mede o uso real de memória física (real memory) pelo processo e **VMS** mede a quantidade de espaço virtual alocado pelo processo, não necessariamente em uso real.

# Processos

Escreva código-fonte em python para determinar, em MB, quanto de memória física e quanto de memória virtual está consumindo o processo cujo PID é igual a 0.

# Processos

Escreva código-fonte em python para determinar, em MB, quanto de memória física e quanto de memória virtual está consumindo o processo cujo PID é igual a 0.

```
import psutil

def obter_memoria_processo(pid):
    try:
        # Obter informações do processo
        processo = psutil.Process(pid)
        memoria_info = processo.memory_info()

        # Converter para MB
        memoria_fisica_mb = memoria_info.rss / (1024 * 1024)  # Resident Set Size
        memoria_virtual_mb = memoria_info.vms / (1024 * 1024)  # Virtual Memory Size

        return memoria_fisica_mb, memoria_virtual_mb
    except psutil.NoSuchProcess:
        return None, None
```

# Processos

Escreva código-fonte em python para determinar, em MB, quanto de memória física e quanto de memória virtual está consumindo o processo cujo PID é igual a 0.

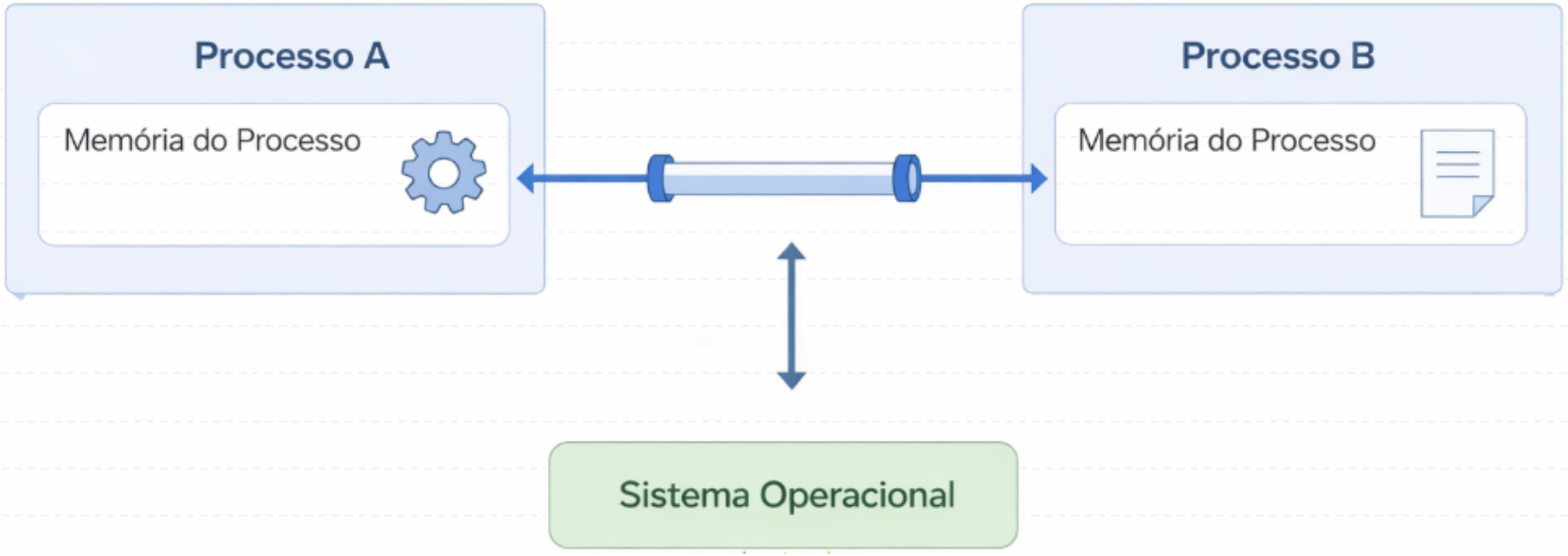
```python
# Definir o PID

pid = 0
memoria_fisica, memoria_virtual = obter_memoria_processo(pid)

if memoria_fisica is not None:
    print(f"Processo PID {pid}:")
    print(f"Memória Física (RSS): {memoria_fisica:.2f} MB")
    print(f"Memória Virtual (VMS): {memoria_virtual:.2f} MB")
else:
    print(f"Processo com PID {pid} não encontrado.")
```

# Processos

Outros parâmetros possíveis são:

| | |
|---|---|
| 11 | io_counters: Contadores de operações de entrada/saída (e.g., leitura/escrita em disco). |
| 12 | num_threads: Número de threads usadas pelo processo. |
| 13 | ppid: ID do processo pai (Parent Process ID). |
| 14 | cwd: Diretório de trabalho atual do processo. |
| 15 | environ: Variáveis de ambiente do processo (requer permissões adequadas). |
| 16 | nice: Prioridade ou niceness do processo. |

A **comunicação entre processos** é o mecanismo que permite que dois ou mais processos troquem informações e sincronizem suas atividades. Como cada processo possui seu próprio espaço de memória, o sistema operacional oferece recursos específicos, como ***pipes***, ***filas de mensagens***, ***memória compartilhada*** e ***sockets***, para viabilizar essa troca de dados de forma controlada e segura.



## Processo A — Produtor (C)

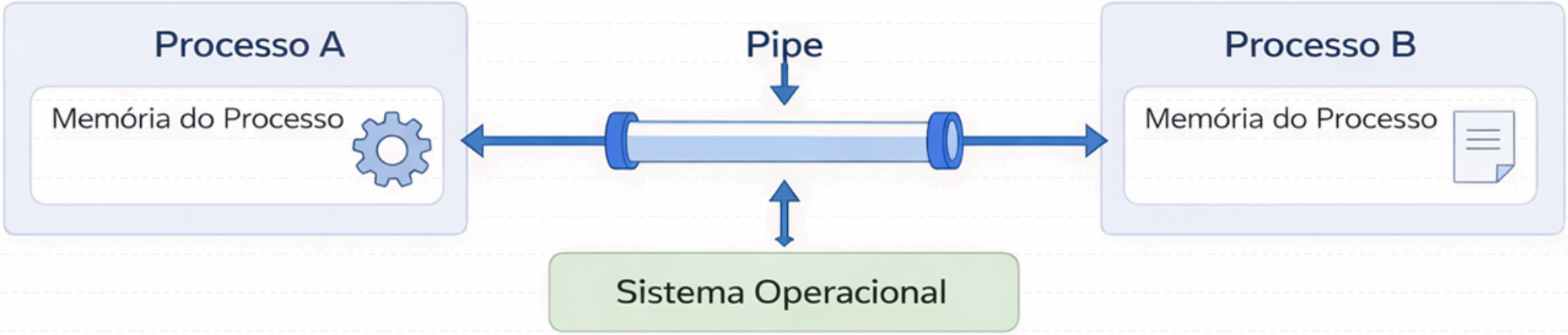
```c
#include <stdio.h>

int main(void) {
    for (int i = 1; i <= 5; i++) {
        printf("mensagem %d\n", i);
        fflush(stdout);
    }
return 0;
}
```

← Garante envio imediatdo pelo pipe

## Processo B — Consumidor (Python)

```python
import sys

for linha in sys.stdin:
    print(f"[Python recbeu] {linha.strip()}")
```

↑ Lé dados recebidos pelo pipe



```c
#include <stdio.h>

int main(void) {
    for (int i = 1; i <= 5; i++) {
        printf("mensagem %d\n", i);
        fflush(stdout);
    }
    return 0;
}
```

```python
import sys

for linha in sys.stdin:
    print(f"[Python recebeu] {linha.strip()}")
```

```json
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "IPC: Compilar e Executar",
      "type": "shell",
      "command": "gcc produtor.c -o produtor.exe; .\\produtor.exe | python consumidor.py",
      "group": {
        "kind": "build",
        "isDefault": true
      },
      "problemMatcher": []
    }
  ]
}
```

O arquivo **tasks.json** é o mecanismo de automação de tarefas do Visual Studio Code, permitindo configurar comandos que serão executados automaticamente pelo terminal integrado do editor. Ele é utilizado para organizar e padronizar processos recorrentes, como compilar programas, executar scripts, encadear comandos com pipes ou rodar testes, evitando que o usuário precise digitar manualmente as mesmas instruções a cada execução. Ao definir tarefas nesse arquivo, o desenvolvedor pode dispará-las por meio de menus ou atalhos de teclado, garantindo maior produtividade, reprodutibilidade e organização do fluxo de trabalho dentro do ambiente de desenvolvimento.

# Processos

Elabore dois programas que se comuniquem por meio de um **pipe do sistema operacional**, utilizando comunicação entre processos (IPC). O primeiro programa, escrito em **Python**, deve atuar como **produtor**, realizando a leitura de um número inteiro informado pelo usuário via teclado e enviando esse valor para a saída padrão. O segundo programa, escrito em **C**, deve atuar como **consumidor**, recebendo o número por meio da entrada padrão (stdin), calculando o **fatorial** do valor recebido e exibindo o resultado na tela. A execução deve ocorrer por meio do encadeamento dos programas com o operador de pipe no terminal (por exemplo, python produtor.py | programaC), demonstrando a comunicação unidirecional entre processos.

```python
import sys


print("Digite um número inteiro:", file=sys.stderr) # Mostra o prompt no stderr (não vai para o pipe)
valor = sys.stdin.readline().strip() # Lê do teclado

# Valida

try:
    n = int(valor)
except ValueError:
    print("ERRO: entrada não é um inteiro.", file=sys.stderr)
    sys.exit(1)

if n < 0:
    print("ERRO: use um inteiro não negativo.", file=sys.stderr)
    sys.exit(1)

print(n) # Envia SOMENTE o número para o stdout (vai para o pipe)
sys.stdout.flush()
```

```c
#include <stdio.h>
#include <stdlib.h>

unsigned long long fatorial(int n) {
    if (n < 0) return 0;
    unsigned long long resultado = 1;
    for (int i = 1; i <= n; i++) resultado *= i;
    return resultado;
}

int main(void) {
    int numero;

    if (scanf("%d", &numero) != 1) {    // Lê da entrada padrão (stdin)
        fprintf(stderr, "Entrada inválida.\n");
        return 1;
    }

    printf("Fatorial de %d = %llu\n", numero, fatorial(numero));
    return 0;
}
```

## Programação concorrente e distribuída

# Processos

Professor Marcelo Eustáquio
(marcelo.junior@p.ucb.br)

**Q01**

Crie um programa que liste todos os processos ativos no sistema, exibindo seus PIDs, nomes e estados (ex.: "running", "sleeping").

*Dica:*

Itere sobre psutil.process_iter() e recupere os atributos pid, name e status.

**Q02**

Desenvolva um script que identifique o processo que está consumindo mais CPU no momento.

*Dica:*

Use psutil.process_iter() para iterar sobre os processos e recupere o atributo cpu_percent() de cada processo. Ordene os resultados e exiba o PID e o nome do processo com maior uso de CPU.

**Q03**

Escreva um programa que monitore continuamente o uso de memória física (RSS) de um processo específico, dado seu PID.

*Dica:*

Use um loop para chamar process.memory_info().rss periodicamente e exibir o valor em MB.

**Q04**

Desenvolva um programa que calcule a soma da memória física (RSS) usada por todos os processos ativos no sistema.

*Dica:*

Itere sobre psutil.process_iter(), recupere memory_info().rss de cada processo e some os valores.

**Q05**

Crie um script que liste todos os processos que estão usando mais de 100 MB de memória física (RSS).

*Dica:*

Filtre os processos onde memory_info().rss / (1024 * 1024) é maior que 100.

**Q06**

Crie um script que liste todos os processos que estão em estado "sleeping".

*Dica:*

Filtre os processos usando proc.info['status'] == psutil.STATUS_SLEEPING.

**Q07**

Desenvolva um programa que monitore o uso de disco (leitura/escrita) de um processo específico, dado seu PID.

*Dica:*

Use psutil.Process(pid).io_counters() para recuperar as estatísticas de leitura/escrita de disco.

**Q08**

Crie um script que encerre um processo específico, dado seu PID.

*Dica:*

Use psutil.Process(pid).terminate() para encerrar o processo. Trate exceções como psutil.NoSuchProcess caso o PID não exista.

**Q09**

Crie um programa que liste todos os processos ativos no sistema, ordenados pelo uso de memória física (RSS), em ordem decrescente. Exiba o PID, nome e uso de memória (em MB) de cada processo.

*Dica:*

Itere sobre psutil.process_iter() para recuperar memory_info().rss de cada processo, ordene os processos pelo valor de RSS e exiba os resultados.

**Q10**

Desenvolva um script que identifique e liste todos os processos que estão consumindo mais de 50% da CPU no momento. Exiba o PID, nome e uso de CPU de cada processo.

*Dica:*

Use psutil.process_iter() para iterar sobre os processos e recupere cpu_percent() de cada processo. Filtre os processos onde cpu_percent() é maior que 50 e exiba os resultados.

**Q11**

Escreva um programa que monitore o uso de rede (bytes enviados e recebidos) de um processo específico, dado seu PID. Exiba os valores periodicamente.

*Dica:*

Use psutil.Process(pid).io_counters() ou psutil.net_io_counters() para recuperar as estatísticas de rede do processo. Atualize os valores em intervalos regulares usando um loop.

## Programação concorrente e paralela

# Processos

Professor Marcelo Eustáquio
(marcelo.junior@p.ucb.br)

# Processos

Na ciência da computação, **processo** é uma instância de um programa em execução em um sistema operacional, podendo ser considerada uma entidade ativa que inclui o código do programa, seus dados, recursos alocados e informações de execução, como registradores e contador de programa.

**Para refletir...**

*Explique como o conceito de processo, enquanto instância de um programa em execução, se relaciona com os mecanismos de isolamento e segurança em sistemas operacionais modernos. Em sua resposta, aborde como o isolamento entre processos protege memória e recursos do sistema, limita o impacto de falhas ou ataques e contribui para a estabilidade e segurança do ambiente computacional.*

# Processos

Em sistemas operacionais modernos, o **conceito de processo** está diretamente ligado aos mecanismos de **isolamento e segurança**, pois cada processo representa uma instância independente de um programa em execução, com **espaço de endereçamento próprio** e **recursos controlados pelo sistema operacional**.

O **isolamento de processos** impede que um processo acesse diretamente a memória, os dados ou o estado interno de outro processo. Esse isolamento é garantido principalmente pelo uso de **memória virtual** e pelo **modo de execução do processador** (modo usuário e modo kernel). Processos em modo usuário não podem executar instruções privilegiadas nem acessar áreas sensíveis do sistema, reduzindo o risco de falhas ou ações maliciosas comprometerem todo o sistema.

Do ponto de vista da **segurança**, cada processo é executado com um **conjunto específico de permissões**, associado ao usuário ou ao contexto de segurança que o iniciou. Assim, mesmo que um processo seja explorado por um ataque (por exemplo, por meio de uma vulnerabilidade em um navegador), seus efeitos ficam restritos ao escopo daquele processo, evitando o comprometimento global do sistema.

Além disso, sistemas modernos utilizam mecanismos adicionais, como **controle de acesso**, **sandboxing**, **containers** e **políticas de segurança obrigatórias** (como SELinux ou AppArmor), que reforçam o isolamento entre processos e limitam ainda mais o que cada um pode fazer. Dessa forma, o conceito de processo funciona como uma **fronteira de proteção**, sendo fundamental para a estabilidade, confiabilidade e segurança dos sistemas operacionais contemporâneos.

# Processos

(Determinando o estado de um processo) Escreva um programa que recebe o PID de um processo e exibe seu estado atual.

*É importante que os estados possíveis são running (em execução), sleeping (dormindo/esperando), disk_sleep (esperando I/O de disco), stopped (parado) e zombie (processo zumbi).*

# Processos

(Determinando o estado de um processo) Escreva um programa que recebe o PID de um processo e exibe seu estado atual (ex.: em execução, parado, suspenso).

```
import psutil

def estado_processo(pid):
    try:
        processo = psutil.Process(pid)
        estado = processo.status()
        return estado
    except psutil.NoSuchProcess:
        return "Processo não encontrado"

# Exemplo de uso

pid = int(input("Digite o PID do processo: "))
estado = estado_processo(pid)
print(f"O estado do processo com PID {pid} é: {estado}")
```

# Processos

(Processos em estado zombie) Escreva um programa que liste todos os processos que estão em estado “zombie”. A seguir, modifique o código-fonte escrito para mostrar tosos os processos que estão parados, ou seja, estão em estado “stopped”.

*É importante que os estados possíveis são running (em execução), sleeping (dormindo/esperando), disk_sleep (esperando I/O de disco), stopped (parado) e zombie (processo zumbi).*

# Processos

(Processos em estado zombie) Escreva um programa que liste todos os processos que estão em estado “zombie”.

```
import psutil

def listar_processos_zumbis():
    zumbis = []
    for processo in psutil.process_iter(['pid', 'name', 'status']):
        if processo.info['status'] == psutil.STATUS_ZOMBIE:
            zumbis.append(processo.info)
    return zumbis

zumbis = listar_processos_zumbis()
if zumbis:
    print("Processos Zumbis:")
    for zumbi in zumbis:
        print(f"PID: {zumbi['pid']}, Nome: {zumbi['name']}")
else:
    print("Nenhum processo zumbi encontrado.")
```

# Processos

(Processos ativos ou não) Escreva um programa para verificar se um processo está ativo.

# Processos

(Processos ativos ou não) Escreva um programa para verificar se um processo está ativo.

```
import psutil

def verificar_processo_ativo(pid):
    return psutil.pid_exists(pid)

pid = int(input("Digite o PID do processo: "))

if verificar_processo_ativo(pid):
    print(f"O processo com PID {pid} está ativo.")
else:
    print(f"O processo com PID {pid} não está ativo ou terminou.")
```